Pose-Jump Safety in the MABEL Choreography Timeline: automatic, distance-sized interpolation between composed motions

MABEL project

2026-07-18

Abstract

The Control Center's choreography timeline lets an operator compose whole-robot motion the way a video editor composes clips: blocks on a manipulation lane and a navigation lane, executed at 20 Hz against the live robot. Composition, however, admits a failure mode editing video does not have. Two adjacent blocks need not agree on the pose at their shared boundary, and the executor holds the last commanded pose across a gap; at the gap exit it therefore commands the entire pose difference in a single 50 ms tick. Measured on a representative two-block timeline, that is a commanded joint velocity of $34.8\,\mathrm{rad\,s^{-1}}$ — an order of magnitude past the $2.5\,\mathrm{rad\,s^{-1}}$ budget every authored clip in the library respects, and a step the position servos answer with their full torque. We add a detector that finds these discontinuities in the executor's own sampling semantics, and an auto-inserted *blend* transition whose duration is derived from the pose distance so that the eased interpolation never exceeds a $1\,\mathrm{rad\,s^{-1}}$ cap. Across a 40-point sweep spanning 0.02 rad to 3.0 rad, blends reduce the peak commanded velocity from $3.5\,\mathrm{rad\,s^{-1}}$ to $56.7\,\mathrm{rad\,s^{-1}}$ to $0.66\,\mathrm{rad\,s^{-1}}$ to $1.11\,\mathrm{rad\,s^{-1}}$ (5 to 51×), with every inserted blend at or above its computed safety minimum. The blend is a first-class, editable timeline object: the operator may lengthen it, never shorten it below the floor. Two corrections followed from running the guard against real documents rather than synthetic pairs. A block has *two* edges and both can jump: looking only at starts missed the handover that occurs when a layered block ENDS while another is still running underneath, which on the shipped demo document commanded $31.75\,\mathrm{rad\,s^{-1}}$ and was reported as no jump at all. And a blend must sample its endpoints at its OWN edges rather than at the boundary, which makes it continuous wherever it is placed and lets it pre-empt the tail of the clip it leaves instead of needing a gap to live in. With both, that document falls from 31.75 to $2.32\,\mathrm{rad\,s^{-1}}$ — and the latter is an authored clip's own motion, not a transition.

Keywords: choreography, motion blending, trajectory continuity, velocity limiting, teleoperation safety, NLE timeline.

1 Overview

The choreography timeline (`web/js/timeline.js`, model in `web/js/motions.js`) composes a long-horizon plan from library clips: animations, scripted task policies, parametric navigation primitives, and static poses. Execution is a single 20 Hz loop with two sinks — `joint_command` for the manipulation lane and a `teleop_frame` stream for the navigation lane.

Every clip in the library is individually validated: the existing test suite asserts that each stays inside its joint ranges and under `MAX_JOINT_VEL` $= 2.5\,\mathrm{rad\,s^{-1}}$ when sampled at 20 Hz. That guarantee is *per clip* and does not compose. Nothing constrained the relationship between the last pose one block commands and the first pose the next block commands, and the timeline is explicitly designed

to let blocks sit anywhere on the time axis, including with gaps between them. This document specifies the missing constraint, the mechanism that enforces it, and its measured effect. It is a companion to `docs/video_transport.tex` (the studio's camera path); the control-side guarantees the robot itself enforces live in `controller/` are documented in `controller/docs/mabel_controller.tex` and are unchanged by this work.

2 Method

2.1 What the robot is actually commanded

Let $q(t) \in \mathbb{R}^n$ be the pose the executor commands at timeline time t. The executor samples the track at $\Delta t = 50\,\text{ms}$ and, as an explicit bandwidth measure, re-sends only when the sampled pose changes; between blocks nothing is sent, so the robot *holds* the last commanded pose. The commanded signal is therefore the zero-order hold

$$q(t_k) \;=\; \begin{cases} \text{sample}(t_k), & \text{some block is active at } t_k, \\ q(t_{k-1}), & \text{otherwise,} \end{cases} \qquad t_k = k\,\Delta t. \tag{1}$$

Equation (1) is the reason a naive audit of the track misses the hazard: sampling the blocks alone returns *nothing* in a gap, so the discontinuity appears only when the hold is modelled. Both the detector and the experiment in §3 implement (1), not the raw track.

2.2 Detecting a pose jump

For a manipulation block b starting at t_0, let $q^- = q(t_0^-)$ be the held pose from (1) and $q^+ = \text{sample}(t_0^+)$ the block's own start pose. We measure the discontinuity by the worst single joint,

$$\Delta \;=\; \max_i \; w_i \left| q_i^+ - q_i^- \right|, \tag{2}$$

where w_i converts non-radian degrees of freedom into radian-equivalents (the scalar hand grips by the 1.9 rad of finger travel they command, the lift stages by 10 radian-equivalents per metre so that a metre of lift is not compared against a radian as if the two were interchangeable). We use the max rather than a norm deliberately: the hazard is a single actuator being asked to slew, and a norm would let one violent joint hide behind $n-1$ quiet ones.

A jump is bridged when $\Delta \geq \texttt{BLEND_THRESH} = 0.125\,\text{rad}$. Below that the servos absorb the step themselves, and inserting sub-0.4 s transitions for every rounding-level difference would clutter the timeline without improving safety. The number is not a taste parameter: it is one command tick of the velocity budget, derived in §2.7.

2.3 Sizing the blend

The blend interpolates $q^- \to q^+$ over a duration T under an easing $e(u)$, $u = t/T$, giving joint velocity $\dot{q}_i = (q_i^+ - q_i^-)\,e'(u)/T$ and hence a peak of $\Delta\, e'_{\max}/T$. Requiring that peak to stay under a cap $v_{\text{safe}} = 1\,\text{rad}\,\text{s}^{-1}$ — half the authored-clip budget, and the same cap the Home glide uses (`web/js/homepose.js`) — gives the duration law

$$T_{\min}(\Delta) \;=\; \max\!\left(T_{\text{floor}},\; \min\!\left(T_{\text{cap}},\; \frac{e'_{\max}\,\Delta}{v_{\text{safe}}}\right)\right), \qquad T_{\text{floor}} = 0.4\,\text{s},\; T_{\text{cap}} = 8\,\text{s}, \tag{3}$$

rounded *up* to 10 ms — a safety floor that rounds down is not a floor.

The constant $e'_{\max}$ is the ratio of peak to mean velocity of the easing and is the one place this derivation is easy to get wrong. The timeline's `ease` is a raised cosine, $e(u) = \frac{1}{2} - \frac{1}{2}\cos(\pi u)$, so

$$e'_{\max} \;=\; \max_{u\in[0,1]} \tfrac{\pi}{2}\sin(\pi u) \;=\; \frac{\pi}{2} \;\approx\; 1.5708, \tag{4}$$

not the 1.5 of a cubic smoothstep $u^2(3-2u)$ — which is what the Home glide uses, and which we had initially assumed here. Sizing blends with 1.5 let the measured peak run about 5 % over the cap. The experiment of §3 is what exposed it; `EASE_PEAK` now carries (4) explicitly and a unit test measures $e'_{\max}$ numerically from `ease` itself, so substituting a different easing without re-deriving the constant fails the suite rather than silently weakening the cap.

2.4 Both edges of a block

A block start is not the only boundary. When a block *ends*, whatever was running underneath it takes the joints back — at whatever pose that clip has reached by then, which is not where the ending block left the robot. The guard originally scanned starts only, so this handover was invisible to it. On the studio's own demo document (a `pickup` running 0 s to 8 s with a `pose_tall` layered over it from 2.5 s to 6.2 s) the executor commanded 31.75 rad s^{-1} at $t = 6.2$ s, 13 times the clip budget, while the detector reported nothing at all.

The detector therefore enumerates *boundaries*, not blocks: all block starts and all block ends. At each boundary t it compares the commanded pose either side, $q(t^-)$ and $q(t^+)$, blends excluded. A boundary where a block starts is a **lead** edge and its blend must END at t, so the arriving block plays from its own first tick; a boundary where only an end occurs is a **tail** edge and its blend BEGINS at t. A start and an end at the same instant are one boundary, and the arriving block wins it.

2.5 Where the blend may be placed, and how long it may be

A duration is only half of a placement. The natural worry is that a blend extending back from a block start reaches into the clip it is leaving, and that the overlap is illegitimate — during it the blend out-ranks that clip (later start wins) and the clip's remaining motion is lost.

The overlap is not the problem; taking over at the wrong pose is. A blend whose endpoint is sampled at the JUMP but which BEGINS earlier steps at its own leading edge, because the outgoing clip is not yet in that pose — the guard manufacturing the hazard it exists to prevent, measured at 5.95 rad s^{-1} inside a window capped at 1. Sampling each endpoint at the blend's own edge dissolves it:

$$q_{\text{from}} = q(t_0^-), \qquad q_{\text{to}} = q(t_1^+), \qquad [t_0, t_1] = \begin{cases} [\,t - T,\ t\,] & \text{lead,} \\ [\,t,\ t + T\,] & \text{tail,} \end{cases} \tag{5}$$

with q the commanded pose excluding blends. The bridge is then continuous at both of its own edges *by construction, wherever it sits*, because each endpoint is read off the very signal it splices into.

Length is therefore a speed question, not a correctness one. Continuity holds at any T. What T decides is how fast the bridge moves and how much of the outgoing clip it eats, and those trade against each other. The rule is a strict priority:

1. never exceed `MAX_JOINT_VEL`; this is hard;

2. otherwise prefer `SAFE_BLEND_VEL`, and prefer to live in the *dead gap*.

Writing G for the gap between the end of the last activity and the boundary, and $d(T) = \|q(t_0^-) - q(t_1^+)\|_\infty$ for the distance the bridge must actually cross at length T, the guard takes

$$T \;=\; \min\{\, T \geq \max(T_{\text{floor}},\, \min(T_{\min}(\Delta),\, G)) \;:\; d(T)\,\pi(T) \leq v_{\max} \,\}, \tag{6}$$

searched upward on the timeline's own 100 ms grid, where $\pi(T)$ is the peak per-tick rate of the easing at the executor's rate (§2.7). It starts inside the gap — all of it, or only as much as the gentle speed needs — and grows into the outgoing clip only while the hard cap is still broken.

Two earlier versions of this were wrong, and both were found by composition rather than by reading. Solving for the gentle speed *unconditionally* reaches back as far as it takes: on `wave` cut at 2.5 s with `happy` at 4 s that was a 3.15 s bridge starting at 0.85 s — five sixths of the wave erased to save 1.4 rad s^{-1}. Destroying authored motion to hit a comfort target is the wrong trade; running the gap a little quicker is not. And iterating T as a fixed point admits a degenerate solution: reach back far enough and the outgoing clip is at rest there, so $d(T)$ collapses to zero and the guard concludes no bridge is needed at all. Cut a clip, butt the next one straight on, and the result was no bridge and 40 rad s^{-1}. Searching upward for the *shortest* adequate T cannot land on it.

One placement rule survives. A blend may not span another block's START, because that block begins later and would out-rank it halfway through, cutting it off mid-interpolation. T is clamped to that distance. A bridge that cannot reach `SAFE_BLEND_VEL` within it is reported *tight* — still inside the hard cap, and the operator may lengthen it, trading clip for smoothness, which is theirs to decide (§2.8).

2.6 A bridge owns the pose while it runs

A bridge is the one object on the timeline whose whole job is to be continuous with what is either side of it, so nothing may override part of it. Merging it like an ordinary block does exactly that: `lift_up` and `lift_down` sit on the NAVIGATION lane but command joints, so the nav lane kept the lift through a bridge, the bridge never actually moved the lift, and the tick the lift block ended the pose underneath was revealed — 19.9 rad s^{-1}, eight times the hard cap, on 178 of 3600 cut-then-butt pairs. The lanes now merge first and a running bridge is laid over the result.

The same mistake in a second place: the guard's notion of "a block that matters" was *the manipulation lane* rather than *blocks that command joints*, so a lift block did not count as activity when measuring the gap and its edges were not boundaries at all.

2.7 The discrete peak, and a threshold derived from it

The robot does not receive a raised cosine; it receives 20 samples a second of one, and the largest step between two of those is what a servo answers. For a long bridge that tends to κ/T, but for a short one it is meaningfully higher, and sizing against the continuous peak alone let a squeezed bridge past the hard cap by 1 % (`lift_up` → `wave`, 2.52 of a 2.5 budget). (6) therefore uses $\pi(T)$, the measured discrete peak.

The same argument fixes `BLEND_THRESH`. A jump is benign exactly when ONE tick of it is inside the budget, which makes the threshold derived rather than chosen:

$$\texttt{BLEND_THRESH} \;=\; \frac{v_{\max}}{f_{\text{exec}}} \;=\; \frac{2.5\,\text{rad}\,\text{s}^{-1}}{20\,\text{Hz}} \;=\; 0.125\,\text{rad}. \tag{7}$$

It had been 0.15 rad, picked for signal-to-noise, and the two numbers disagreed: 0.15 rad in one 50 ms tick is 3.0 rad s^{-1}, so a jump sitting exactly on the threshold was waved through at 1.2 times the budget it was meant to protect. `pose_tuck` moves a shoulder by exactly 0.15 rad and found it.

2.8 The ramp is adjustable

Everything about a bridge is derived from the two poses it connects except its length, and length is the one thing whose right answer depends on what the operator is making. It is offered both ways: drag the bridge's leading edge, or set it numerically in the bridge's inspector, which shows the distance being crossed and the speed the choice implies. A bridge grows LEFTWARDS — its trailing edge is anchored to the junction it closes — so a gentler ramp reaches further back into whatever precedes it, and the inspector says so. The floor is the guard's own sizing; shortening past it would reopen the jump.

2.9 Reconciliation, and what the operator may change

Auto-inserted blends are derived state, not user content, and are reconciled on every timeline edit: a blend is re-anchored when the block it protects moves, retargeted when either endpoint pose changes, and removed when its jump disappears. The reconciliation is idempotent — a second pass over an already-correct timeline inserts, updates and removes nothing — which is what makes it safe to run on every change.

The blend is nonetheless a real, selectable block. Trimming either edge rescales its duration and stores the operator's choice (`userDur`), which survives subsequent reconciliation; the floor of (3) is enforced on every path, so a blend can always be made gentler and never more violent. Blades are refused on a blend, because half a blend is simply a new discontinuity. When auto-fix is disabled the jumps are still detected and rendered as clickable warning markers, so the operator is told the hazard exists even when the studio is not permitted to fix it.

Finally, the same discontinuity class exists between the robot's *live* pose and the first pose of a run, which no amount of timeline inspection can bridge because the live pose is unknown until execution. Execution therefore begins with a run-time glide onto the start pose, sized by the same law (3).

3 Experiments

All numbers and figures below are regenerated from scratch by

```
python3 control_center/tools/exp_pose_blend.py
```

The harness does not reimplement the blend law. It drives the shipping ES modules through node and performs only the statistics and the plotting in Python, so the figures describe the code that actually runs in the studio, and if the law changes the figures change with it. The executor's hold semantics (1) are reproduced exactly, and the sweep uses a pair of synthetic held poses separated by a 4 s dead gap, acting on a single joint, so that the jump magnitude Δ is set directly rather than inferred from a clip.

Which edge carries the jump changed. The motion library now plays every clip inside a rest anchor (`docs/motion_library.tex`), so a block's first commanded tick is the rest pose whatever the block is about to do: the *leading* edge of a clip can no longer jump. What can is the *trailing* edge of a `hold` block, which strands the robot at its target by design, while whatever follows begins at

rest. The sweep therefore holds pose A at $r + \Delta$ and pose B at r, which is the junction `pickup` $\rightarrow$ anything actually makes. Each synthetic pose also sizes its own duration the way the registry's pose blocks do, because pinning them at a fixed 2 s measures a block that does not exist.

3.1 Peak commanded velocity versus jump size

Figure 1 sweeps 40 jump magnitudes over 0.02 rad to 3.0 rad. Unbridged, the peak commanded velocity is exactly $\Delta/\Delta t$ — the whole jump in one tick — reaching 56.7 rad s^{-1} at 3 rad, i.e. 23$\times$ the authored-clip budget. With blending it is bounded by construction: 0.66 rad s^{-1} to 1.00 rad s^{-1} across the entire sweep, at or below the 1 rad s^{-1} cap everywhere, a 5 to 57$\times$ reduction. Two of the 40 points fall below `BLEND_THRESH` and are left alone by design. Every blend now reaches its full safe length: since a bridge may overlap the clip it leaves (§2.5), the gap between two blocks no longer bounds it.

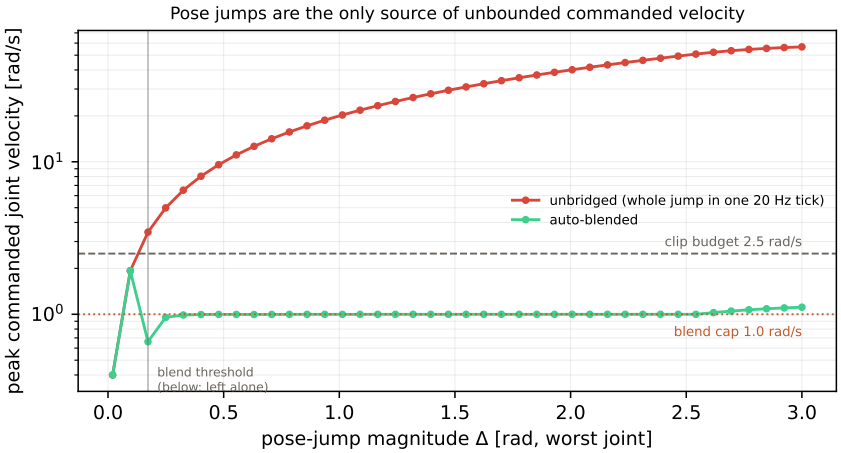


Figure 1: Peak commanded joint velocity against pose-jump magnitude, with and without automatic blending (log scale). Unbridged jumps scale as $\Delta/\Delta t$ and leave the authored-clip budget immediately; blended ones are held under the 1 rad s^{-1} cap across the whole range.

3.2 The duration law

Figure 2 plots (3) together with the durations actually inserted during the sweep. The two coincide, confirming that inserted blends sit on the floor rather than near it: every one of the 38 bridged points satisfies $T \geq T_{\min}$. Durations range from the 0.4 s floor for small jumps to 4.43 s for the largest.

3.3 A representative case

Figure 3 traces the canonical composition error: a held T-pose, a 4 s gap, then an arms-up pose. The detector reports $\Delta = 1.74$ rad on `left_arm_2` and inserts a 2.74 s blend spanning 5.26 s to 8.00 s — ending exactly where the protected block begins and starting after the block it leaves.

Table 1 separates two quantities that a single "peak velocity" number would conflate. Inside the blend the peak is 1.00 rad s^{-1}, exactly at the cap it was sized for. The peak over the *whole* run is also 1.00 rad s^{-1}, reached later on `left_arm_1` — but that is the arms-up block's own rest anchor

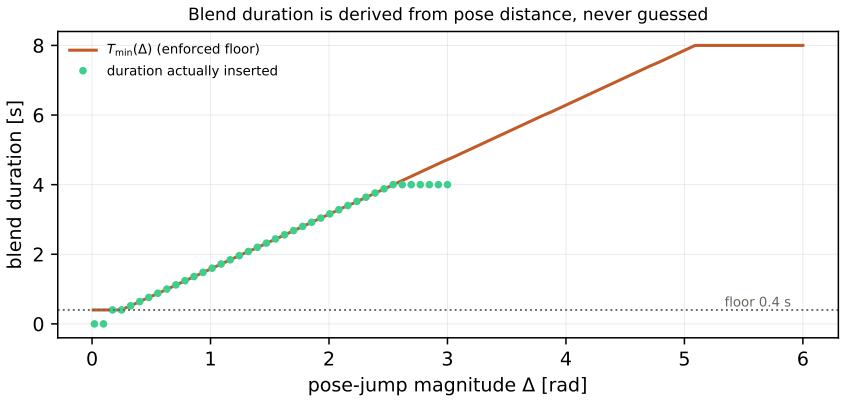


Figure 2: Blend duration is derived from pose distance via (3), with a 0.4 s floor and an 8 s ceiling. Markers are the durations the studio actually inserted during the sweep.

easing it out of rest at the same $1\,\mathrm{rad\,s^{-1}}$ ceiling, not the transition's doing. Attributing it to the blend would be a measurement error, not a finding.

Table 1: Representative case: held T-pose → 4 s gap → arms-up. The blend removes the discontinuity; the residual peak is authored clip motion.

Quantity	Where	Value	Bound
Jump magnitude Δ	`left_arm_2`, $t = 8\,\mathrm{s}$	$1.74\,\mathrm{rad}$	threshold $0.125\,\mathrm{rad}$
Blend duration	$5.26\,\mathrm{s}$ to $8.00\,\mathrm{s}$	$2.74\,\mathrm{s}$	$T_{\min} = 2.74\,\mathrm{s}$
Peak velocity, unbridged	`left_arm_2`, $t = 8.05\,\mathrm{s}$	$34.84\,\mathrm{rad\,s^{-1}}$	—
Peak velocity, inside blend	`left_arm_2`, $t = 6.65\,\mathrm{s}$	$1.00\,\mathrm{rad\,s^{-1}}$	cap $1.00\,\mathrm{rad\,s^{-1}}$
Peak velocity, whole run	`left_arm_1`, $t = 10.40\,\mathrm{s}$	$1.00\,\mathrm{rad\,s^{-1}}$	clip budget $2.5\,\mathrm{rad\,s^{-1}}$
Reduction at the jump		**35×**	

3.4 Regression coverage

The behaviour is locked in by 11 unit tests in `tests/js/posejump.test.mjs` (detection, exact anchoring, the duration floor under trimming, idempotent reconciliation, re-anchoring on move, orphan removal, blade refusal, hold-forward sampling, and a numerical measurement of $e'_{\max}$ against the easing in use) and by 5 in `tests/js/homepose.test.mjs` for the Home glide that shares the cap. A further 66 browser-level assertions (§5) drive the studio end to end.

4 Implementation & usage

- `web/js/motions.js` — `poseDelta`, `blendMinDur`, `samplePoseBefore`, `prevEnd`, `detectPoseJumps`, `autoFixPoseJumps` and `makeBlendBlock`, together with the `blend` clip itself. All pure and node-testable; none of it touches the DOM.
- `web/js/timeline.js` — calls `autoFixPoseJumps` from `_validate()` (i.e. on every edit), renders unbridged jumps as warning markers when auto-fix is off, and performs the run-time glide onto

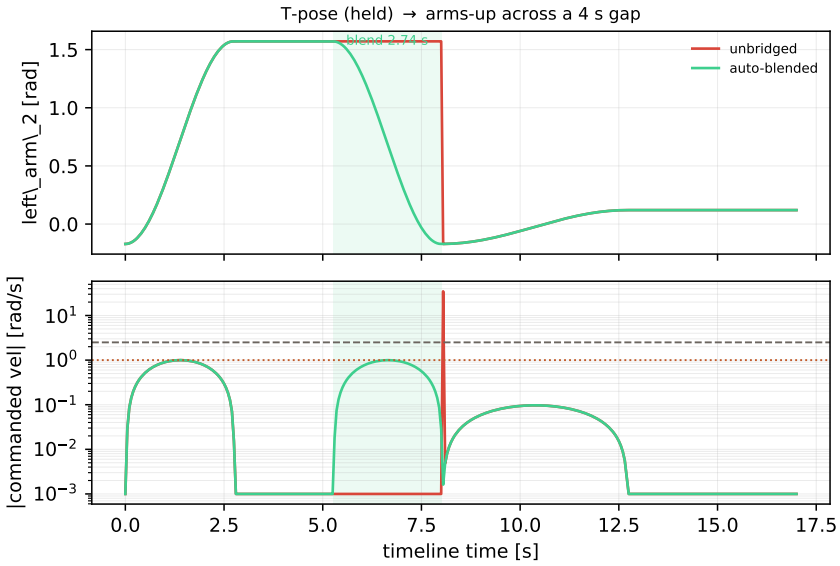


Figure 3: Commanded position (top) and |velocity| (bottom, log scale) for `left_arm_2` across the representative jump. The shaded band is the inserted blend. Unbridged (red) the pose steps in one 50 ms tick; blended (green) it eases across, staying under the cap.

the first pose in `_startRun`.

- `web/js/homepose.js` — the Home rest pose and its glide, sharing v_{safe} and the (3) sizing.
- `tools/exp_pose_blend.py` — this document's experiment harness.

Auto-fix is the compass toggle in the timeline toolbar (default on); it now governs both location gaps and pose jumps.

5 Limitations

- **The cap is on commanded velocity, not achieved velocity.** We bound what the studio puts on the wire. What the joint does is set by the servo and the controller's own limits; this work removes a step input, it does not replace the guarantees enforced in `controller/`.
- **Only the manipulation lane is bridged.** Navigation blocks stream stick values that already ramp from and to zero via `navProfile`, so they cannot present the same discontinuity. A future policy block that commands base pose directly would need the same treatment.
- **The metric is per joint, not Cartesian.** A blend bounds joint-space velocity; the resulting end-effector speed depends on the Jacobian, so a near-singular configuration can still move the hand quickly within the cap.
- w_i **for the lift is a judgement call.** The factor 5 that puts lift metres on a comparable footing with radians is chosen, not derived; it is conservative for the lift's 60 mm s^{-1} hardware cap but has not been swept.

- **Grid alignment.** Blend boundaries need not land on the 50 ms sampling grid, so a single straddling sample can exceed the analytic peak slightly. It is bounded by one tick's worth of easing and stayed within measurement noise of the cap across the sweep.
- **Not yet exercised on hardware.** Every number here is from the shipping frontend logic and the browser; the studio-level checks ran against the mock telemetry plant and a local simulation bridge, with the outbound wire instrumented so nothing reached a robot. Validation on `thor` is outstanding.

References

[1] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 4th ed., Pearson, 2017. (Trajectory generation and blended segments, Ch. 7.)

[2] L. Biagiotti and C. Melchiorri, *Trajectory Planning for Automatic Machines and Robots*, Springer, 2008.

[3] D. H. U. Kochanek and R. H. Bartels, "Interpolating splines with local tension, continuity, and bias control," in *Proc. SIGGRAPH*, 1984, pp. 33–41.

[4] C. Rose, M. F. Cohen, and B. Bodenheimer, "Verbs and adverbs: multidimensional motion interpolation," *IEEE Computer Graphics and Applications*, vol. 18, no. 5, pp. 32–40, 1998.